Automated Data Pipelines & Private Cloud Services

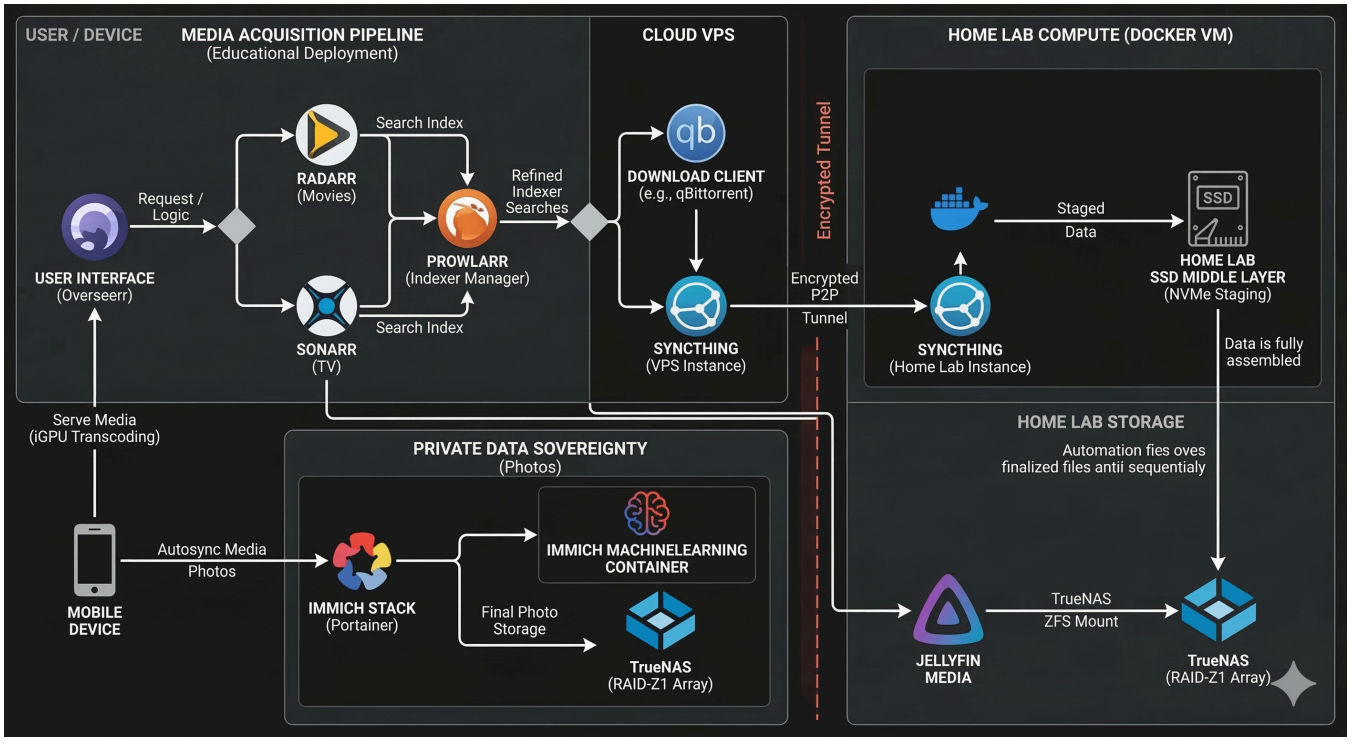
Project Overview

A robust infrastructure is only as valuable as the services it hosts. This phase of the project focused on deploying highly available, containerized applications to manage data acquisition, media streaming, and private data sovereignty. By replacing third-party cloud dependencies (like Google Photos or streaming services) with self-hosted alternatives, I established complete control over my data privacy and processing logic.

Architecture & Orchestration: The Dedicated Docker VM

To ensure strict resource isolation, I did not run containers directly on the hypervisor or the storage appliance.

1. **Dedicated Compute Node:** I provisioned a dedicated Ubuntu Server VM within Proxmox explicitly for container workloads.
2. **Container Orchestration:** I deployed **Portainer** to manage the Docker environment.
3. **Logical Isolation (Stacks):** Instead of running a single monolithic `docker-compose` file, I separated services into logical **Stacks** within Portainer (e.g., `media-stack`, `management-stack`, `photo-stack`). This microservice approach ensures that an update or crash in the media pipeline does not impact the photo backup infrastructure.



The Distributed Acquisition Pipeline

To maintain network security and optimize I/O performance, I engineered a distributed, multi-node data pipeline. The workflow operates asynchronously across cloud and local environments:

1. **Request & Logic (Overseerr ➔ Sonarr/Radarr ➔ Prowlarr):** A centralized frontend (Overseerr) handles user requests, which are routed to logic engines (Sonarr/Radarr) and indexer managers (Prowlarr) running in local Docker containers.
2. **Off-site Secure Ingestion (VPS Cloud):** To protect my home IP address and offload heavy bandwidth ingestion, the actual data retrieval occurs on a hardened, off-site Virtual Private Server (VPS).
3. **Asynchronous Synchronization (Syncthing):** Syncthing monitors the VPS via an encrypted peer-to-peer tunnel and securely pulls the downloaded data back to my local network.
4. **The SSD Middle Layer (IOwait Mitigation):** To prevent crippling my mechanical TrueNAS drives with random write operations, Syncthing downloads the data to a high-speed NVMe SSD staging area first.
5. **Storage & Presentation (TrueNAS ➔ Jellyfin):** Once the file is fully assembled on the SSD, it is sequentially written to the TrueNAS ZFS array. Finally, Jellyfin (utilizing iGPU hardware acceleration) serves the media to end-user devices.

Private Data Sovereignty: Immich & Local AI

To reclaim my personal data from public cloud providers, I deployed **Immich**, a high-performance, self-hosted photo and video backup solution.

- **Automated Ingestion:** Mobile devices automatically synchronize assets to the server the moment they connect to the home network (or via the WireGuard VPN).
- **Localized Machine Learning (ML):** Instead of sending biometric data to external servers, the Immich stack includes a dedicated Machine Learning container. This container utilizes the server's compute resources to perform localized facial recognition, object detection, and metadata extraction, ensuring that all AI processing remains strictly on-premise.

Engineering Challenges & Solutions

Challenge 1: The Multi-Node I/O Bottleneck

Initially, I configured Syncthing to write data directly from the VPS to the TrueNAS NFS share. Because Syncthing writes in small, random chunks, it overwhelmed the mechanical hard drives (HDDs), causing a massive spike in IOwait that locked up the entire Proxmox host.

The Fix: I re-architected the storage path. I mapped a high-speed NVMe SSD virtual disk to the Syncthing container. The data is staged on the SSD, and only upon completion do the Radarr/Sonarr automation scripts securely move the finalized, sequential file over to the TrueNAS HDD array.

Challenge 2: Cross-Stack Container Networking

Because I split my services into multiple Portainer Stacks (e.g., separating the `media-stack` from the `proxy-stack`), containers could not communicate with each other by default.

The Fix: I designed and implemented a custom, external Docker bridge network. By defining `network: external` within my YAML configurations, I allowed services like Jellyfin and Sonarr to communicate securely across stack boundaries using internal DNS resolution, without exposing unnecessary ports to the host network.

What I Learned

Building this pipeline was a masterclass in **DevSecOps** and **Systems Architecture**.

- I learned how to securely bridge cloud infrastructure (VPS) with on-premise hardware using encrypted sync protocols.
- Managing the Immich ML container taught me how to provision and monitor AI workloads, balancing CPU inference limits against standard application memory requirements.
- Most importantly, resolving the Syncthing storage bottleneck reinforced the principle that in systems engineering, understanding the physical hardware limitations is just as critical as writing clean code.